

UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela



Proyecto 1

Juan Cruz

Diego Morales

Erick Guerra

GUATEMALA, 9 de septiembre del 2026

Introducción

Este informe describe el desarrollo de un programa que dibuja en pantalla varios elementos animados con movimiento y física, y que luego se mejoró para aprovechar varios núcleos del procesador al mismo tiempo. El proyecto se realizó como parte del curso de Computación Paralela y Distribuida, con el objetivo de aplicar de forma práctica los conceptos de programación paralela con memoria compartida usando la herramienta OpenMP.

El programa recibe la cantidad de elementos a dibujar como parámetro, junto con otras opciones como el tamaño de la ventana y la cantidad de hilos a usar. Cada elemento se mueve por la pantalla, rebota al chocar con los bordes, choca de forma elástica con los demás elementos, gira sobre sí mismo y cambia de tamaño de forma cíclica. Además, se dibujan líneas entre los elementos que están cerca unos de otros, formando una especie de constelación, y se deja una estela de movimiento en vez de borrar la pantalla en cada cuadro.

El trabajo se dividió en dos partes principales. Primero se construyó una versión secuencial, en la que todo el trabajo lo hace un solo hilo, y que sirve como punto de comparación. Después se construyó una versión paralela, en la que el cálculo de la física de los elementos se reparte entre varios hilos con OpenMP. Con ambas versiones funcionando, se midió cuánto tiempo tarda cada una en distintos escenarios, y se calculó el speedup y la eficiencia obtenidos al paralelizar el programa.

En las siguientes secciones se explica el contexto teórico detrás de OpenMP y la programación paralela, el diseño de las dos versiones del programa, la metodología usada para decidir qué partes paralelizar y cómo, los resultados de las mediciones de rendimiento, y las conclusiones obtenidas a partir de ese trabajo.

Antecedentes

Durante muchos años, la forma más común de hacer que un programa corriera más rápido era simplemente esperar a que salieran procesadores con una velocidad de reloj más alta. Sin embargo, desde hace tiempo esa mejora se ha vuelto mucho más lenta, y en su lugar los fabricantes empezaron a poner varios núcleos dentro de un mismo procesador. Esto significa que, para aprovechar todo el poder de cómputo disponible, un programa ya no puede limitarse a ejecutar sus instrucciones una detrás de otra: necesita dividir su trabajo entre varios núcleos que lo ejecuten al mismo tiempo.

A esta forma de programar se le llama programación paralela, y uno de sus modelos más usados es el de memoria compartida, en el que varios hilos de un mismo programa trabajan a la vez y todos pueden leer y escribir las mismas variables en memoria. Este modelo es distinto al de memoria distribuida, donde cada proceso tiene su propia memoria y se comunican enviándose mensajes entre sí. El proyecto desarrollado en este informe usa el modelo de memoria compartida, ya que todos los hilos trabajan dentro de un mismo proceso y acceden directamente a la misma lista de elementos en memoria.

OpenMP es una de las herramientas más utilizadas para programar con memoria compartida en lenguajes como C y C++. En vez de exigir que el programador reescriba todo el programa usando hilos manejados a mano, OpenMP funciona por medio de directivas que se agregan encima del código secuencial ya existente. El compilador se encarga de traducir esas directivas en la creación y el manejo de los hilos. Esto permite ir paralelizando un programa de forma gradual, empezando por las partes más sencillas y avanzando poco a poco hacia las más complejas, tal como lo describen Chapman, Jost y Van Der Pas (2008) en su libro sobre programación paralela con memoria compartida usando OpenMP.

Entre las directivas más usadas están la de repartir las iteraciones de un ciclo for entre varios hilos, y las que sirven para proteger el acceso a datos que varios hilos podrían modificar al mismo tiempo. Cuando dos o más hilos pueden llegar a modificar la misma variable de forma simultánea, se dice que existe una condición de carrera, y el resultado del programa deja de ser confiable si no se protege ese acceso. La especificación oficial de OpenMP describe con detalle estos mecanismos de sincronización, entre ellos las secciones críticas y las operaciones atómicas, que se usaron en este proyecto para evitar ese tipo de errores.

Por último, antes de decidir qué parte de un programa paralelizar y cómo hacerlo, conviene seguir un método ordenado en vez de agregar directivas de forma improvisada. Uno de los métodos más conocidos para esto es PCAM, que significa Partición, Comunicación, Aglomeración y Mapeo, propuesto por Foster (1995). Este método guía el diseño de un programa paralelo dividiendo el problema en tareas pequeñas, identificando qué comunicación necesitan esas tareas entre sí, agrupándolas de forma eficiente y finalmente asignándolas a los hilos disponibles. En este proyecto se usó PCAM como guía para decidir cómo repartir el trabajo de la física del programa entre los hilos.

Desarrollo del proyecto

Descripción general del programa

El programa recibe todos sus parámetros por línea de comandos, sin ningún valor escrito directamente dentro del código. El parámetro obligatorio es la cantidad de elementos a dibujar, y de manera opcional se puede indicar el ancho y el alto de la ventana, la cantidad de hilos a usar en la versión paralela, la semilla para generar los números aleatorios, y varios valores que ajustan la apariencia, como el tamaño de los elementos, su rapidez, la opacidad de la estela y la distancia máxima para dibujar una línea entre dos elementos. Antes de crear la ventana, el programa revisa que todos estos valores tengan sentido, y si algo está mal muestra un mensaje explicando el error sin abrir ninguna ventana.

Cada elemento se representa como un pequeño polígono de color, con posición, velocidad, radio, ángulo de rotación, velocidad angular y una fase que controla su pulso de tamaño. En cada cuadro de animación, el programa mueve todos los elementos, revisa y resuelve los choques entre ellos, calcula qué pares de elementos están lo bastante cerca como para unirlos con una línea, y por último dibuja todo en pantalla junto con un reloj y algunos contadores de información.

Diseño de la versión secuencial

La versión secuencial fue la primera en construirse, y sirve como punto de comparación para medir después qué tanto mejora la versión paralela. En esta versión, un solo hilo se encarga de todo el trabajo: mover los elementos, resolver los choques, calcular las líneas de conexión y dibujar cada cuadro.

Una parte importante del diseño es que la lectura de los argumentos de línea de comandos y la creación de los elementos iniciales son exactamente el mismo código que después usa la versión paralela. De esta manera, si se corren las dos versiones con los mismos parámetros y la misma semilla, ambas empiezan desde el mismo estado inicial y hacen el mismo trabajo, lo cual es necesario para que la comparación de tiempos entre ellas sea justa.

Cada cuadro de animación en la versión secuencial se calcula en tres pasos, siempre en el mismo orden. Primero se mueve cada elemento según su velocidad y se revisa si rebotó contra algún

borde de la ventana. Después se revisan todos los pares posibles de elementos para saber si dos de ellos están chocando, y en caso de que sí, se corrige su velocidad y su posición como si hubiera ocurrido un choque elástico. Por último, se vuelven a revisar los mismos pares de elementos, esta vez solo para saber cuáles están lo bastante cerca como para dibujar una línea entre ellos.

Metodología PCAM aplicada al diseño paralelo

Antes de paralelizar la física del programa se aplicó el método PCAM, que consiste en cuatro pasos.

En la partición, el problema se dividió pensando en cada elemento por separado. El paso de mover los elementos se puede dividir elemento por elemento, ya que cada uno se actualiza de forma independiente. En cambio, los pasos de resolver choques y de calcular las líneas de conexión se dividieron por pares de elementos, ya que ahí es donde está el trabajo real.

En la comunicación, se identificó que mover un elemento no necesita comunicarse con nada más, porque cada uno depende únicamente de sus propios datos. En cambio, cuando dos elementos chocan, sí hace falta actualizar la velocidad y la posición de ambos de forma conjunta, y eso exige proteger ese acceso para que dos hilos no lo hagan al mismo tiempo sobre el mismo elemento. Algo parecido ocurre al armar la lista de líneas de conexión, ya que todos los hilos terminan escribiendo en la misma lista compartida.

En la aglomeración, en vez de tratar cada elemento como una tarea separada para un hilo distinto, lo cual generaría demasiados hilos y mucho gasto extra por crearlos y coordinarlos, se agruparon las iteraciones de cada ciclo for y se dejó que OpenMP repartiera esos grupos entre los hilos disponibles.

Por último, en el mapeo se decidió con qué política repartir esos grupos de iteraciones entre los hilos. Para el paso de mover los elementos se usó un reparto uniforme, ya que cada elemento cuesta prácticamente lo mismo de calcular. Para los pasos de resolver choques y calcular líneas, en cambio, se usó un reparto dinámico, ya que la cantidad de trabajo de cada elemento no es uniforme: el trabajo depende de cuántos elementos quedan por revisar después de él, así que los primeros elementos tienen mucho más trabajo que los últimos. Con un reparto dinámico, cada hilo recibe un nuevo grupo de iteraciones apenas termina el anterior, en vez de recibir de una sola

vez una porción fija que podría dejarlo con mucho más o mucho menos trabajo que los demás hilos.

Versión paralela y mecanismos de sincronización

La versión paralela parte exactamente del mismo diseño que la secuencial, y la diferencia está en cómo se calcula cada uno de los tres pasos de la física por cuadro. El programa que dibuja en pantalla se dejó igual en las dos versiones, ya que la librería usada para dibujar no admite ser usada desde varios hilos al mismo tiempo. Por esa razón, el tiempo que se compara entre la versión secuencial y la paralela es el tiempo dedicado únicamente a calcular la física, y no el tiempo total del programa, que también incluye el tiempo de dibujar cada cuadro.

El paso de mover los elementos se reparte entre hilos sin necesitar ningún mecanismo de protección, ya que cada hilo trabaja únicamente con su propio elemento y no toca los datos de ningún otro.

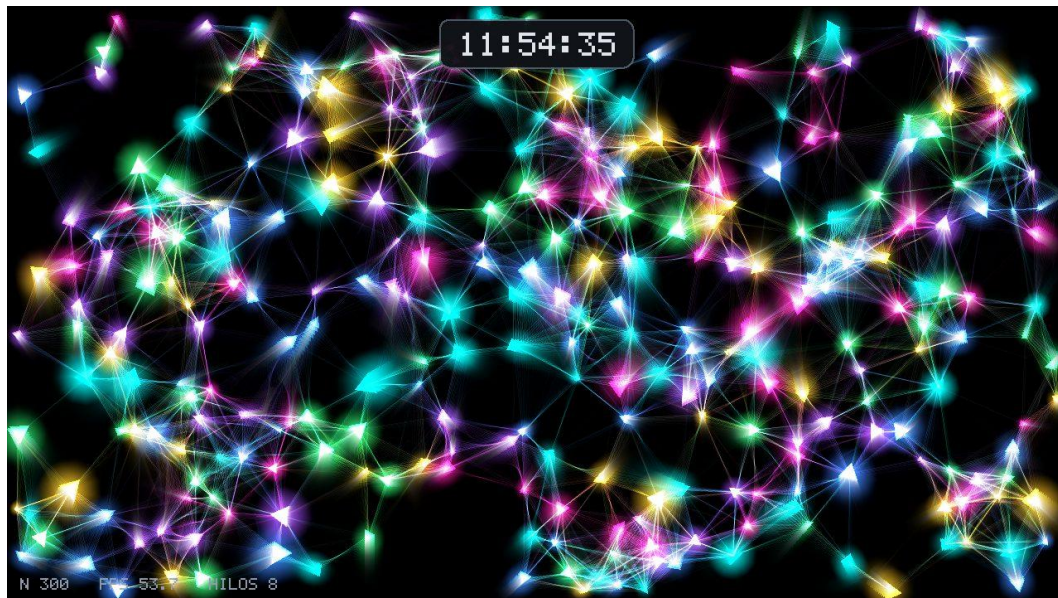


Figura 1. Versión paralela corriendo con trescientos elementos y ocho hilos.

El paso de resolver los choques necesita protección, porque dos hilos podrían estar resolviendo dos choques distintos que involucren al mismo elemento al mismo tiempo. Para evitar que uno sobrescriba los cambios del otro, la actualización de la velocidad y la posición de los elementos

que chocan se protege con una sección crítica. Se prefirió una sección crítica en vez de una operación atómica porque en un mismo choque se deben actualizar varias variables de dos elementos distintos de forma conjunta y consistente, y una operación atómica solo protege una única variable a la vez.

El paso de calcular las líneas de conexión no modifica los elementos, solo lee su posición, así que ahí no hay riesgo de que dos hilos se pisen al leer los mismos datos. El único recurso compartido es la lista final de líneas. Para evitar que todos los hilos entren a una sección crítica cada vez que encuentran una línea nueva, lo cual sería muy costoso, cada hilo primero guarda sus propias líneas en una lista privada, y solamente al terminar se junta esa lista privada con la lista compartida, dentro de una sola sección crítica por hilo.

Resultados de las mediciones de desempeño

Para medir el speedup y la eficiencia se corrieron ambas versiones del programa con los mismos parámetros y la misma semilla, de manera que las dos recorrieran exactamente la misma cantidad de trabajo. Cada prueba se repitió diez veces y se usó el promedio de esas diez mediciones. El speedup se calculó como el tiempo promedio de la versión secuencial dividido entre el tiempo promedio de la versión paralela, y la eficiencia se calculó dividiendo ese speedup entre la cantidad de hilos usados.

Tabla 1. Speedup y eficiencia obtenidos en las tres pruebas realizadas.

Prueba	Cantidad de elementos	Hilos	Speedup	Eficiencia
1	1000	4	1.30x	32.4%
2	1000	8	1.33x	16.6%
3	3000	8	2.55x	31.9%

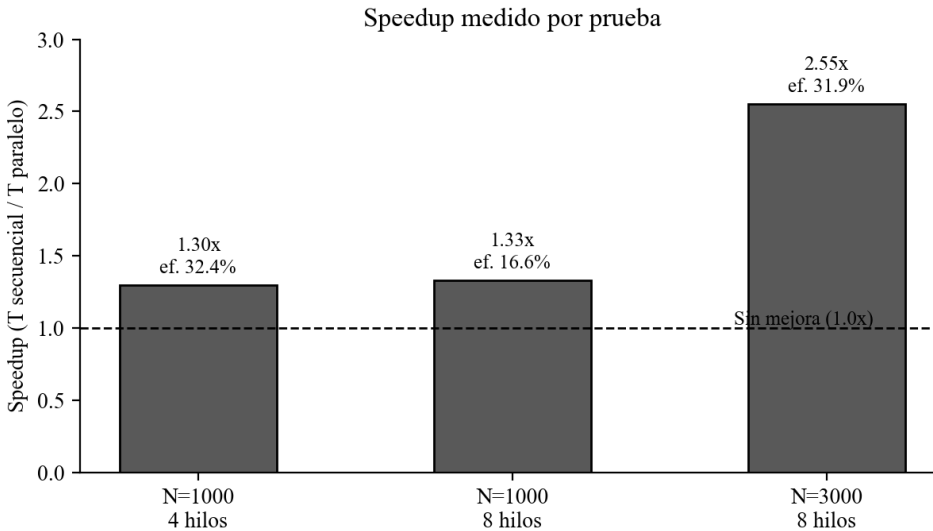


Figura 2. Speedup medido en cada prueba, comparado contra la línea de referencia sin mejora.

Con mil elementos, el speedup obtenido fue bajo, de apenas 1.30 veces con cuatro hilos y 1.33 veces con ocho hilos. Además, al pasar de cuatro a ocho hilos la eficiencia cayó a la mitad, lo cual muestra que agregar más hilos no sirvió de mucho en este caso. Esto ocurre porque, con pocos elementos, el trabajo de calcular la física es pequeño, y el costo de crear y coordinar los hilos de OpenMP termina siendo comparable al trabajo que en realidad se está ahorrando.

Con tres mil elementos y ocho hilos, en cambio, el speedup subió a 2.55 veces, casi el doble que con mil elementos usando la misma cantidad de hilos. Esto confirma que los pasos de resolver choques y calcular líneas de conexión, que revisan todos los pares posibles de elementos, crecen mucho más rápido que el costo fijo de coordinar los hilos a medida que aumenta la cantidad de elementos. En otras palabras, entre más elementos tiene que procesar el programa, más conviene repartir ese trabajo entre varios hilos.

Optimización adicional explorada

Además del diseño ya descrito, se probó una alternativa distinta para proteger los choques entre elementos, con la idea de mejorar todavía más el speedup. En vez de usar una sola sección crítica compartida por todos los pares de elementos, se probó asignarle un candado propio a cada elemento, de forma que dos choques que no compartieran ningún elemento pudieran resolverse al mismo tiempo sin esperarse entre sí.

Esta alternativa se midió contra la versión con una sola sección crítica, usando tres mil elementos y ocho hilos, que es el caso con más choques ocurriendo al mismo tiempo. El resultado fue que el candado por elemento no mejoró el tiempo de la versión ya existente, sino que resultó levemente más lento. La razón es que la mayor parte del costo de resolver los choques está en revisar la distancia entre cada par de elementos, un cálculo que ya se hace en paralelo sin ningún tipo de protección, y no en la parte que sí se protege, que solamente se ejecuta cuando dos elementos están chocando de verdad. Como los choques reales son una fracción pequeña de todos los pares revisados, la sección crítica compartida casi nunca hace que un hilo tenga que esperar a otro, y agregar un candado por elemento solo sumó costo extra sin quitar ninguna espera real.

Como esta alternativa no logró una mejora medible frente a la versión ya implementada, no se reporta como una mejora adicional del proyecto, sino como parte del proceso de diseño, ya que ayudó a confirmar que la sincronización escogida es adecuada para este problema. La forma más clara de mejorar el speedup en un trabajo futuro no sería cambiar el mecanismo de sincronización, sino cambiar la forma en que se buscan los pares de elementos cercanos, reemplazando la revisión de todos los pares posibles por una división del espacio en celdas, de manera que solo se comparen elementos que están cerca unos de otros.

Conclusiones

1. La paralelización no siempre produce una mejora grande cuando la cantidad de elementos es pequeña. Con mil elementos, el tiempo de la versión secuencial y el de la paralela fueron parecidos, porque el costo de coordinar los hilos es comparable al trabajo que se está repartiendo entre ellos.
2. El tiempo de dibujar en pantalla domina sobre el tiempo de calcular la física. Para mil elementos, dibujar cada cuadro tomó mucho más tiempo que calcular su movimiento y sus choques, lo cual confirma que paralelizar solamente la física tiene un efecto limitado sobre el tiempo total del programa mientras la cantidad de elementos sea baja.
3. Los pasos que revisan todos los pares posibles de elementos son los que más se benefician de repartirse entre varios hilos. Con una cantidad mayor de elementos, esos pasos crecen mucho más rápido que el costo de coordinar los hilos, y es ahí donde el speedup se vuelve realmente significativo.

Recomendaciones

- Repetir las mediciones de speedup con una cantidad de elementos mayor a cinco mil, para observar con más claridad el punto en el que el trabajo paralelo compensa por completo el costo de coordinar los hilos.
- Explorar una estructura de datos basada en una grilla espacial, que divida el canvas en celdas y solo compare elementos que están en celdas cercanas, ya que esto reduciría de forma importante la cantidad de pares que se revisan en cada cuadro.
- Repetir las mediciones en otra computadora con una cantidad distinta de núcleos disponibles, ya que el número de hilos que conviene usar depende del hardware donde corre el programa.

Referencias

1. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. Recuperado de <https://www.openmp.org/specifications/>

2. Blaise Barney, Lawrence Livermore National Laboratory. (2021). *Introduction to Parallel Computing*. Recuperado de <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>
3. ***Parallel computing using OpenMP***. Medium. Recuperado de <https://medium.com/jungletronics/parallel-computing-using-openmp-0b61c5c9d2e6>

Anexo 1. Diagrama de flujo del programa

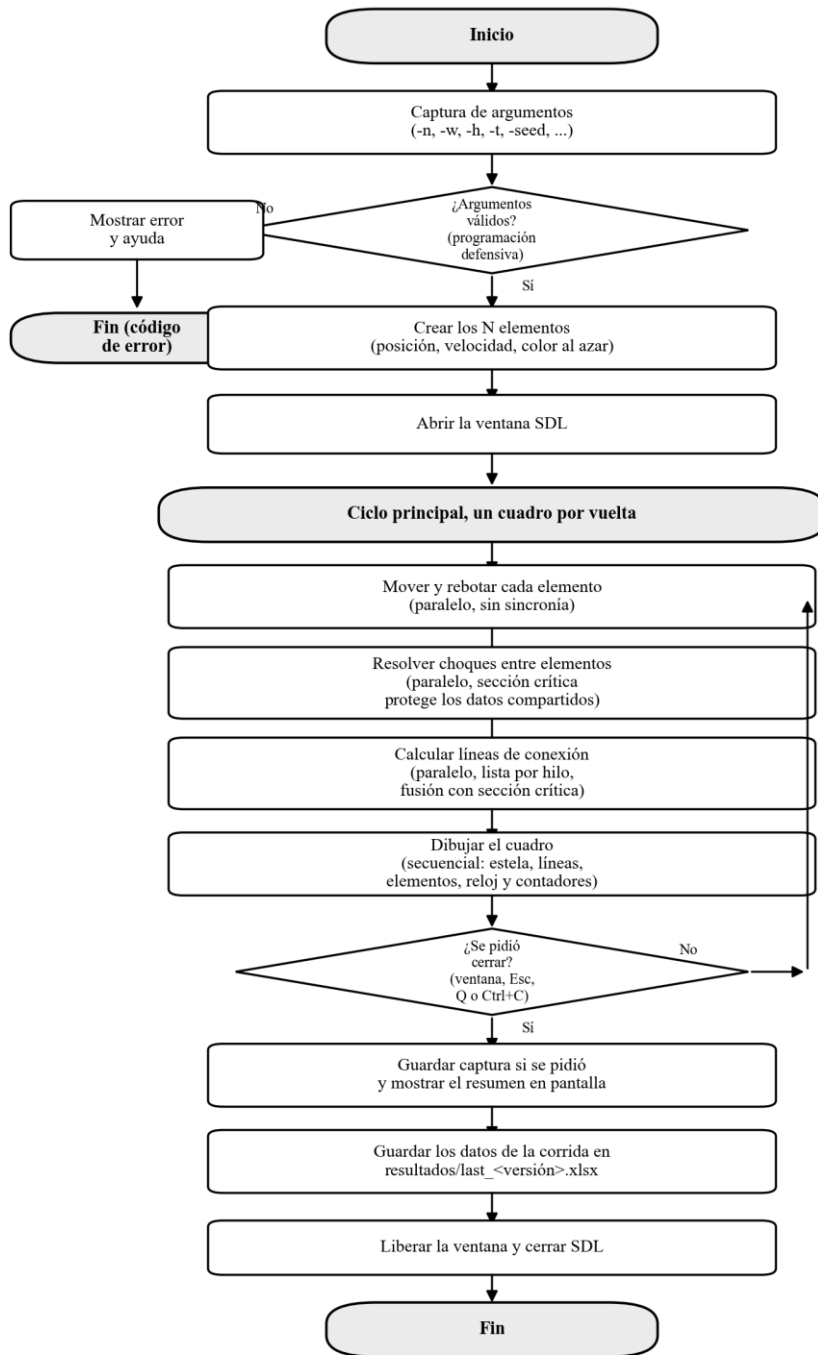


Figura 1. Diagrama de flujo del programa, desde la captura de argumentos hasta el despliegue de resultados.

Anexo 2. Catálogo de funciones

A continuación, se describe cada función pública del programa: qué recibe, qué devuelve y qué hace. Las funciones están agrupadas según el archivo donde están definidas.

src/common/args.cpp: parseArgs

Campo	Detalle
Firma	bool parseArgs(int argc, char** argv, Config& config)
Entradas	argc es la cantidad de argumentos recibidos. argv es la lista de esos argumentos como texto. config es la estructura donde se guardan los valores ya leídos.
Salida	Devuelve verdadero si todos los argumentos son válidos, y falso si falta alguno, no es un número o está fuera de rango. En ese caso explica el error por la salida de error del programa.
Descripción	Recorre los argumentos recibidos y llena la configuración según las banderas encontradas, como la cantidad de elementos, el tamaño de la ventana, la semilla y la cantidad de hilos. Al final revisa que los valores tengan sentido en conjunto, por ejemplo que el radio máximo no sea menor que el mínimo.

src/common/args.cpp: printUsage

Campo	Detalle
Firma	void printUsage(const char* programName)
Entradas	programName es el nombre del programa, normalmente tomado de argv[0].
Salida	No devuelve ningún valor. Imprime el mensaje de ayuda en pantalla.
Descripción	Muestra la lista completa de banderas disponibles junto con una breve descripción y su valor por defecto. Se llama automáticamente cuando parseArgs encuentra un error.

src/common/elements.cpp: createSimulation

Campo	Detalle
Firma	Simulation createSimulation(const Config& config)
Entradas	config contiene los parámetros de la corrida, en especial la cantidad de elementos, la semilla y los rangos de radio y velocidad.
Salida	Devuelve una estructura con los elementos ya creados y la lista de líneas de conexión vacía.
Descripción	Genera cada elemento con una posición, velocidad, ángulo y color al azar, usando la semilla indicada. Si dos corridas usan la misma semilla, obtienen exactamente el mismo estado inicial, lo cual permite comparar los tiempos de las dos versiones de forma justa.

src/secuencial/physics.cpp y src/paralelo/physics.cpp: stepSimulation

Campo	Detalle
Firma	void stepSimulation(Simulation& sim, const Config& config)
Entradas	sim es el estado actual de la animación, que se modifica. config trae los parámetros necesarios, como el paso de tiempo y la distancia máxima para dibujar una línea.
Salida	No devuelve ningún valor. Al terminar, sim queda con las posiciones, velocidades y líneas de conexión actualizadas.
Descripción	Avanza la simulación un paso, llamando primero a la función que mueve los elementos, después a la que resuelve los choques y por último a la que calcula las líneas de conexión. El orden es importante, porque cada paso trabaja sobre el resultado del anterior.

src/common/render.cpp: pollQuitRequest

Campo	Detalle
Firma	bool pollQuitRequest()
Entradas	No recibe ningún parámetro.
Salida	Devuelve verdadero si el usuario pidió cerrar el programa, ya sea con el botón de la ventana, la tecla Escape, la tecla Q o presionando Ctrl más C en la terminal.
Descripción	Revisa los eventos pendientes de la ventana en cada cuadro, sin detener el programa mientras lo hace.

src/common/render.cpp: Renderer::init

Campo	Detalle
Firma	bool Renderer::init(const Config& config)
Entradas	config trae el ancho, el alto y si se debe activar la sincronización vertical.
Salida	Devuelve verdadero si la ventana se pudo crear correctamente.
Descripción	Prepara la librería gráfica, abre la ventana y crea los recursos necesarios para dibujar. Si algo falla, explica el motivo por la salida de error.

src/common/render.cpp: Renderer::drawFrame

Campo	Detalle
Firma	void Renderer::drawFrame(const Simulation& sim, const Config& config, const Metrics& metrics)
Entradas	sim es el estado actual de los elementos. config trae las opciones de apariencia. metrics trae los datos que se muestran en el contador de pantalla.
Salida	No devuelve ningún valor. El resultado se ve directamente en la ventana.

Descripción	Dibuja un cuadro completo: primero la estela, después las líneas de conexión, luego cada elemento, y por último el reloj y los contadores de información en pantalla.
-------------	---

src/common/metrics.cpp: printMetrics

Campo	Detalle
Firma	void printMetrics(const Config& config, const Metrics& metrics, const char* versionName)
Entradas	config y metrics traen los datos de la corrida. versionName indica si la corrida fue de la versión secuencial o de la paralela.
Salida	No devuelve ningún valor. Imprime el resumen en pantalla.
Descripción	Muestra un resumen legible del tiempo total, el tiempo de física, el tiempo de dibujo y los cuadros por segundo, además de una línea de datos que puede usarse para armar tablas de resultados.

src/common/metrics.cpp: saveLastRun

Campo	Detalle
Firma	bool saveLastRun(const Config& config, const Metrics& metrics, const char* versionName)
Entradas	Los mismos datos que recibe printMetrics.
Salida	Devuelve verdadero si el archivo se pudo guardar.
Descripción	Guarda los datos de la corrida más reciente de cada versión en un archivo de Excel, para poder revisarlos con doble clic sin tener que copiar los números a mano desde la terminal.

Anexo 3. Bitácora de pruebas

Todas las pruebas de esta bitácora se corrieron con la misma semilla y la misma cantidad de cuadros por prueba en las dos versiones, de manera que ambas recorrieran exactamente la misma trayectoria y realizaran la misma cantidad de trabajo. Cada prueba se repitió diez veces y se promediaron los resultados. Los archivos con las diez mediciones de cada prueba se encuentran en la carpeta resultados del repositorio del proyecto.

Tabla 1. Resumen de las tres pruebas de rendimiento, con el tiempo de física promediado sobre diez corridas.

Prueba	Elementos	Hilos	Tiempo secuencial promedio	Tiempo paralelo promedio	Speedup	Eficiencia
1	1000	4	0.849 s	0.654 s	1.30x	32.4%
2	1000	8	0.849 s	0.638 s	1.33x	16.6%
3	3000	8	0.462 s	0.181 s	2.55x	31.9%

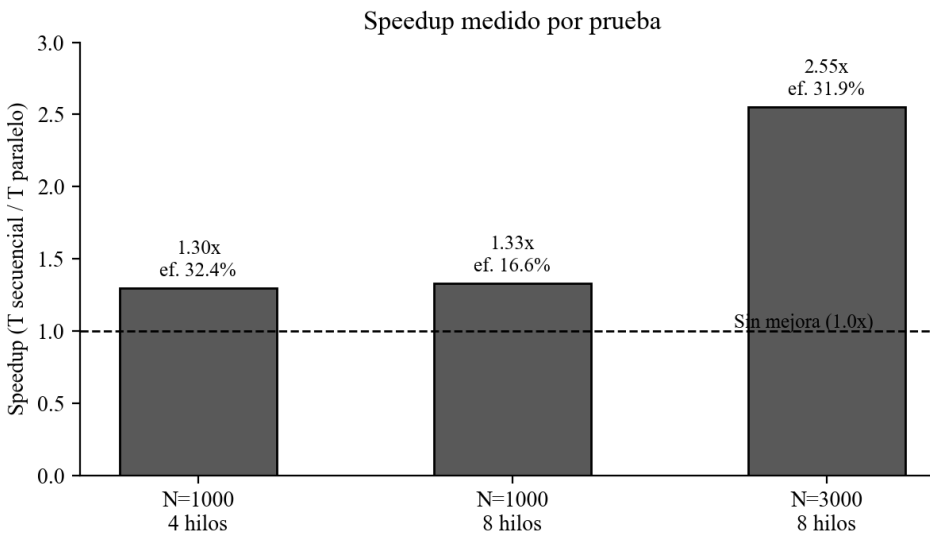


Figura 2. Speedup medido en cada prueba, comparado contra la línea de referencia sin mejora.

A continuación se muestran dos capturas del programa en funcionamiento, tomadas con trescientos elementos una de la versión secuencial y otra de la versión paralela corriendo con ocho hilos.

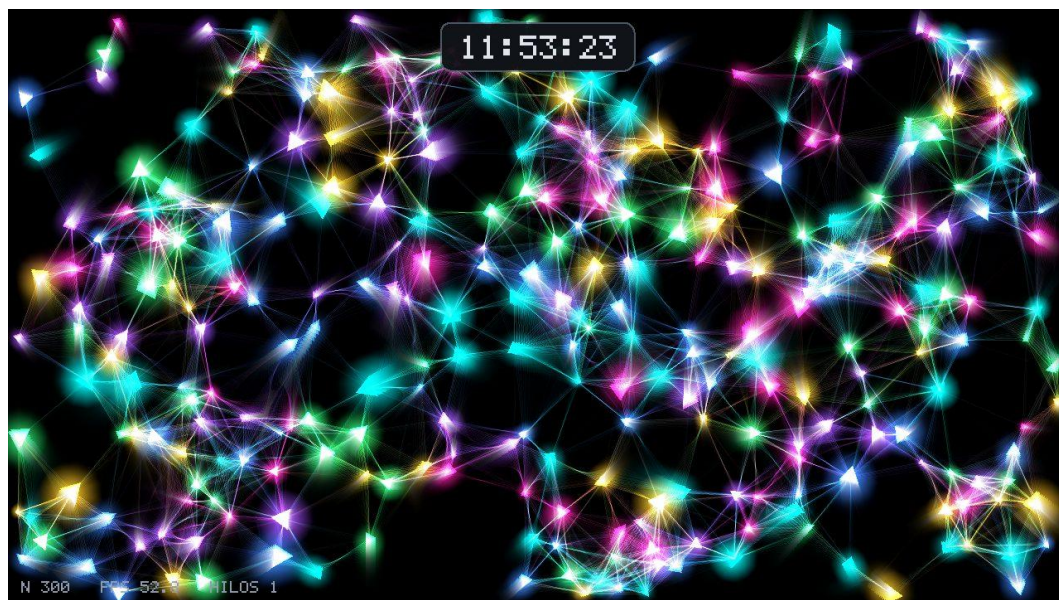


Figura 3. Versión secuencial corriendo con trescientos elementos.

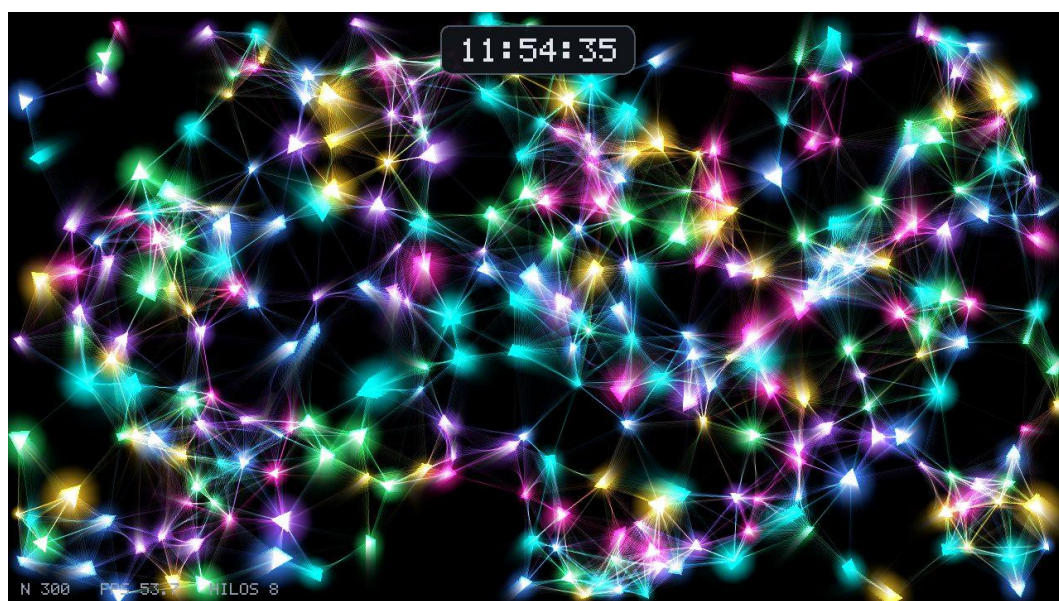


Figura 4. Versión paralela corriendo con trescientos elementos y ocho hilos.